

InfraFlow Frontend Architecture

A codebase-grounded technical reference for the InfraFlow visual AWS infrastructure builder — covering component architecture, state management, routing, the API layer, forms, styling, conventions, known risks, and a debugging quick-reference.

Generated from a direct, file-by-file investigation of `D:\infra as a code\src`. Every claim in this document was verified by reading the actual source — nothing here is inferred from naming conventions or assumed from "typical" React project structure. Where something could not be confirmed with full confidence, it is marked *in italics*.

Revision 2 — updated after a Chapter 9 remediation pass fixed 9 of the original 12 findings (see the callout after the table of contents for a full changelog). Facts specific to those fixes were updated in place throughout; everything else reflects the original investigation and has not been re-verified since.

Table of Contents

1. High-Level Overview
2. Component Architecture
3. State Management
4. Routing
5. API / Data Layer
6. Forms & Validation
7. Styling Approach
8. Known Patterns & Conventions
9. Identified Risks / Technical Debt
10. Debugging Quick-Reference

What Changed Since the First Edition

A remediation pass worked through the Chapter 9 risk list one item at a time, with a full type-check and a real-browser smoke test (all 10 dashboard views, zero console errors) after every change. This is what moved:

Chapter 9 finding	Status	What actually changed
#1 DashboardShell.tsx monolith	Partially resolved	5 self-contained pages (AgentPage, ConnectAwsPage, SupportPage, SuperAdminPage, TerraformPage) extracted to <code>src/dashboard/pages/</code> , plus a new shared <code>src/dashboard/components/DashPrimitives.tsx</code> for <code>Panel / EmptyState</code> . File went from 4,985 → 3,439 lines. The more interdependent pieces (Overview/KPI widgets, Deployments detail views, the builder shell) were deliberately left for a follow-up pass — see Chapter 2.
#2 Zero error boundaries	Resolved	New <code>src/components/ErrorBoundary.tsx</code> wraps the whole app in <code>main.tsx</code> . Verified live: a real render crash in the extracted SuperAdminPage (from a test-harness mock mismatch, not a code bug) was caught cleanly with a fallback UI instead of a white screen.
#3 ~10 duplicated API request helpers	Resolved	New shared <code>src/utils/apiClient.ts</code> (<code>apiRequest / apiFormRequest</code>). All API modules except <code>authClient.ts</code> , <code>ticketApi.ts</code> 's blob fetch, and <code>terraformPayloadApi.ts</code> (each has a genuinely different contract) now delegate to it.
#4 agentApi.ts hardcoded localhost fallback	Resolved	Turned out to be an actual bug, not just a risky fallback: <code>/agent/*</code> is served by the same main backend as everything else (confirmed via <code>routes/index.js</code>), not a separate port-4001 service. Now uses the shared client/base URL like every other module.
#5 No diagram autosave	Partially resolved	New <code>isDirty</code> flag on the Zustand store plus a <code>beforeunload</code> guard in <code>VisualBuilderPage</code> — refreshing or closing the tab with unsaved changes now prompts first. This is a warning, not autosave; the store still has no persist middleware, so a confirmed navigation away still discards everything.
#6 Dashboard navigation doesn't integrate with browser history	Resolved	<code>goToDashboardPage / goToResourceInfo</code> switched from <code>history.replaceState</code> to <code>pushState</code> , plus a new <code>popstate</code> listener. Verified live: back/forward now correctly step through Overview → Deployments → Security and the sidebar's active state stays in sync.
#7 Inconsistent ticket form validation	Resolved	Submit button now disabled until subject+description are filled (matching AuthPage/ConnectAwsPage); the stated 10MB-per-file limit is now actually enforced client-side on both the new-ticket and reply file pickers, with a specific error naming the rejected file(s).
#10 Tailwind configured but unused	Resolved	Not just left dead — actually removed. <code>tailwind.config.js</code> and <code>postcss.config.js</code> deleted, the <code>@tailwind</code> directives dropped from <code>styles.css</code> , and the <code>tailwindcss / postcss / autoprefixer</code> packages uninstalled (63 packages removed). Verified with a full production build.
#8, #9, #11, #12	Unchanged	#8 (required-ness computed, not declared) and #9 (JSON editor allows non-JSON) are by-design, not bugs — left as documentation. #11 and #12 were already-fixed historical notes in the first edition.

ALSO REMOVED IN THIS PASS, NOT FROM CHAPTER 9

The "Node.js Runtime Lab" teaching demo page was deleted outright (frontend page + component + API client, backend route + controller-logic + utils), confirmed dead — its sidebar entry was already commented out and grep found zero other references anywhere in the codebase after removal. `RuntimeLabDetailModal` and the `RuntimeLabDetail` type were kept, since `KpiGrid` and `ResourceTable` still use them independently for their own detail popups. Security Review was *not* removed — an initial request to delete it was corrected after confirming it's a live, linked, real-data-backed page, not dead code like Runtime Lab.

High-Level Overview

What this application does

InfraFlow is a visual AWS infrastructure builder and operations dashboard. A user drags AWS service "nodes" (Lambda, S3, RDS, VPC, IAM roles, API Gateway, etc.) onto a canvas, wires them together with connections, and the app both validates the resulting architecture and generates real Terraform from it. From the same dashboard the user can connect a real AWS account (via an IAM role), deploy the generated Terraform against it (a separate Express backend actually shells out to `terraform apply`), monitor deployments, browse cost/security insights pulled live from AWS, chat with an AI "Well-Architected" agent, build and sync a CI/CD pipeline to GitHub, and file support tickets. A super-admin console exists for user/role/credit management.

Tech stack & major dependencies

Area	Library	Notes
UI framework	React 18.3.1	Function components + hooks throughout; no class components found.
Language	TypeScript 5.7.2	<code>"type": "module"</code> , strict-ish typing via <code>src/types.ts</code> .
Build tool	Vite 6.0.3	<code>npm run build = tsc && vite build</code> (type-checks before bundling).
State management	Zustand 4.5.5	Exactly one store, <code>src/store/diagramStore.ts</code> . No Redux, no MobX.
Canvas / diagramming	reactflow 11.11.4	Powers the entire visual builder (nodes, edges, minimap, controls).
Code editor	@monaco-editor/react + monaco-editor	Used in exactly one place: the Application Pipeline page's workflow/file preview.
Icons	lucide-react 0.468.0	Used everywhere, including a dynamic icon-by-name lookup for AWS service nodes.
Diagram export	html-to-image	PNG/SVG export from the Toolbar.
Routing	None	No react-router or equivalent. Routing is 100% hand-rolled off <code>window.location</code> . See Chapter 4.
Data fetching	Raw <code>fetch()</code>	No axios, no React Query/SWR — but as of Revision 2, a shared <code>src/utils/apiClient.ts</code> factors out the common request/error-unwrap logic. See Chapter 5.
Forms	None	No react-hook-form/formik/yup. Every form is plain <code>useState</code> + manual checks. See Chapter 6.
Styling	Hand-written CSS	One 14,601-line <code>src/styles.css</code> is the styling layer. Tailwind (as of Revision 2: removed — see Chapter 7) is no longer part of the build.

VERIFIED, NOT ASSUMED

The "Tailwind is unused," "no context API usage," and "no error boundaries" (the last since fixed — see the Revision 2 changelog) findings were each confirmed via exhaustive greps across the entire `src/` tree, not by sampling one or two files — see Chapters 7 and 8 for the exact methodology.

Folder structure (Revision 2)

```
src/
├─ App.tsx      Root component: manual pathname routing, auth guard, theme state
├─ main.tsx     ReactDOM entry — wraps <App/> in StrictMode + ErrorBoundary + ReactFlowProvider
├─ styles.css   The application's stylesheet (14,601 lines) — Tailwind removed, this is it now
├─ theme.ts     Theme mode types + helpers (dark / light / pearl)
├─ types.ts     Shared TypeScript types (AwsNode, AwsField, AwsService, edges, etc.)
├─ vite-env.d.ts Vite's ambient type declarations
└─
```

- └─ store/
 - └─ diagramStore.ts The one Zustand store – canvas/diagram state, undo/redo, isDirty flag (new)
- └─ components/ Shared, reusable UI for the visual builder
 - └─ Canvas.tsx React Flow canvas host
 - └─ PropertiesPanel.tsx Node/edge inspector + config-field editor
 - └─ DeploymentModal.tsx Deploy/redeploy flow + status polling
 - └─ Sidebar.tsx Draggable AWS service palette
 - └─ Toolbar.tsx Undo/redo, zoom, export, deploy, save
 - └─ StatusBar.tsx Bottom bar: node/edge counts, validation health
 - └─ AppLogo.tsx Trivial logo component
 - └─ ErrorBoundary.tsx NEW – top-level render-crash fallback, wraps <App/> in main.tsx
 - └─ ResourceInfoViewer.tsx Post-deploy resource output viewer
 - └─ nodes/ Custom React Flow node renderers (AwsServiceNode, GroupBoxNode, LabelNode)
 - └─ edges/ Custom React Flow edge renderer (FlowEdge)
- └─ dashboard/
 - └─ DashboardShell.tsx Dashboard shell + page router + remaining inline pages – 3,439 lines (was 4,985)
 - └─ agentApi.ts AI agent chat API client (fixed: now uses the real shared backend, not a bogus port-4001 URL)
 - └─ applicationPipelineApi.ts CI/CD pipeline API client
 - └─ awsApi.ts AWS account / insights / IAM-role API client
 - └─ dashboardData.ts Static nav config + demo/static fallback content (runtime-lab entries removed)
 - └─ deployRolePolicy.ts IAM trust/permission policy text builders
 - └─ diagramApi.ts Saved-diagram CRUD API client
 - └─ notificationApi.ts Notification bell API client
 - └─ superAdminApi.ts Super-admin console API client
 - └─ ticketApi.ts Support ticket API client (+ TICKET_MAX_ATTACHMENT_BYTES, new)
 - └─ components/
 - └─ DashPrimitives.tsx NEW – shared Panel/EmptyState, used by DashboardShell.tsx and every extracted page below
 - └─ pages/ NEW – pages extracted out of DashboardShell.tsx in the Revision 2 pass
 - └─ AgentPage.tsx AI Well-Architected chat (141 lines)
 - └─ ConnectAwsPage.tsx AWS IAM role connection form (170 lines)
 - └─ SupportPage.tsx Ticket inbox + thread + new-ticket modal (422 lines)
 - └─ SuperAdminPage.tsx User/role/credit admin console (402 lines)
 - └─ TerraformPage.tsx Static Terraform file/code viewer (45 lines)
- (Note: nodeLabApi.ts and the "Runtime Lab" page/route were deleted outright, not extracted – confirmed dead: sidebar entry already commented out, zero other references after removal.)
- └─ auth/
 - └─ AuthPage.tsx Login / register / forgot / reset password UI
 - └─ authClient.ts Auth API client + localStorage session helpers
- └─ landing/
 - └─ LandingPage.tsx Public marketing page (many sub-sections)
 - └─ landingConfig.ts Static copy/content for the landing page
- └─ reference/
 - └─ ReferenceDocsPage.tsx Static reference-doc viewer (embeds a backend-served PDF)
- └─ settings/
 - └─ GithubSettingsPage.tsx GitHub connection management UI
- └─ github/
 - └─ githubApi.ts GitHub OAuth/repo/branch API client
- └─ data/
 - └─ awsServices.ts The AWS service catalog: every service's fields, defaults, terraform type
 - └─ enterpriseDemoSource.ts Loads a static demo Terraform file as a starter diagram
- └─ utils/
 - └─ apiClient.ts NEW – shared apiRequest()/apiFormRequest() used by most API modules
 - └─ exportTerraform.ts Frontend Terraform-preview generator (mirrors the backend generator)
 - └─ importDiagram.ts Parses uploaded/pasted Terraform back into diagram nodes/edges
 - └─ resourceRequirements.ts Hand-written per-service required/recommended/sensitive field schema
 - └─ validate.ts Diagram/architecture linter (not a form validator)
 - └─ deploymentApi.ts Deployment lifecycle API client
 - └─ terraformPayloadApi.ts "Validate" button API client

- └─ terraformValidation.ts Client-side Terraform text sanity checks
- └─ accessControl.ts Role/plan-based feature-gating helpers
- └─ analytics.ts Google Analytics event helpers
- └─ deploymentPlan.ts Small deployment-plan shape helpers

DELETED IN REVISION 2 — NO LONGER PART OF THE TREE

tailwind.config.js , postcss.config.js , src/dashboard/nodeLabApi.ts , and the backend's nodeLabRoutes.js / nodeRuntimeLab.js / tools/nodeConceptLabRunner.js .

Component Architecture

UPDATED IN REVISION 2

`src/dashboard/DashboardShell.tsx` was originally 4,985 lines / 85 top-level functions covering roughly ten unrelated product areas in one file. Five self-contained pages (Terraform export, AI agent chat, AWS-connect form, ticket support, super-admin console) have since been extracted to `src/dashboard/pages/` — see the Revision 2 changelog. It's now **3,439 lines** and still the page router plus the remaining, more interdependent pages (overview + its KPI/resource widgets, the visual builder host, deployments + its detail-view subsystem, application pipelines, security, connect-aws's sibling pages) — a meaningful reduction, not a full resolution. The "Node.js runtime lab" teaching demo mentioned in the original edition of this paragraph no longer exists at all (deleted, not extracted — see the changelog).

2.1 Shell & top-level routing

Component	File	Purpose	Props	Key state
App	App.tsx	Root component. Reads <code>window.location.pathname</code> and manually branches to the right page. Runs the auth guard for protected routes. Owns theme.	—	<code>isProtectedRouteAllowed</code> , <code>isValidatingProtectedRoute</code> , <code>theme</code>
DashboardShell	dashboard/DashboardShell.tsx:195	Dashboard layout: sidebar nav, topbar (search, AWS status pill, notification bell, theme toggle), content area. Its own internal "router" over ~12 dashboard pages via a <code>?view=</code> query param.	<code>theme</code> , <code>onToggleTheme</code>	<code>activePage</code> , <code>awsAccounts</code> , <code>awsInsights</code> , <code>awsRegions</code> , notification state

2.2 Visual builder

The builder is a set of sibling components that coordinate purely through the shared `useDiagramStore()` Zustand store — they do not render each other, and mostly do not pass diagram data as props to one another.

```
VisualBuilderPage (DashboardShell.tsx:647, ~380 lines)
├─ DeploymentModal      (shown instead of the builder shell when deploying)
├─ dashboard-builder-shell
│   ├─ Toolbar           undo/redo, zoom, mode/view switch, export, deploy, save
│   ├─ workspace
│   │   ├─ Sidebar       draggable AWS service palette
│   │   ├─ Canvas        React Flow host (AwsServiceNode / GroupBoxNode / LabelNode / FlowEdge)
│   │   └─ PropertiesPanel positioned popup inspector, shown on node/edge selection
│   └─ StatusBar         node/edge counts + validation health dot
```

Component	File	Notes
Canvas	components/Canvas.tsx	No props — reads everything from <code>useDiagramStore()</code> . Local state: <code>isNodeMoving</code> . Hand-rolled drag-rectangle multi-select via refs. Renders React Flow's Background/Controls/MiniMap/Panel plus the custom node/edge types.
PropertiesPanel	components/PropertiesPanel.tsx (1,007 lines)	Node/edge inspector, positioned via <code>useLayoutEffect</code> . DOM-rect measurement. Contains 3 inline modal sub-components (<code>JsonFieldModal</code> , <code>IamRolePickerModal</code> , <code>Modal</code>) and the zip-upload <code>FilePathField</code> widget, all defined in the same file rather than split out.

DeploymentModal	components/DeploymentModal.tsx	Props: <code>nodes</code> , <code>edges</code> , <code>issues</code> , <code>onClose</code> , <code>onValidate</code> , <code>updateDeploymentId?</code> . Polls deployment status every 2.2s. Handles both "new deployment" and "update existing" flows, plus stuck-deployment force-destroy.
Sidebar	components/Sidebar.tsx	Props: <code>isCollapsed?</code> , <code>onToggleCollapsed?</code> , <code>user?</code> . Local state: <code>search query</code> . Locks services per role/plan via <code>accessControl.ts</code> .
Toolbar	components/Toolbar.tsx	Props: <code>theme/fullscreen/deploy/save</code> callbacks. Handles undo/redo, zoom, export (PNG/SVG via html-to-image, JSON/HCL), Terraform import, auto-layout, and the "Validate" action (posts to the backend).
StatusBar	components/StatusBar.tsx	No props — pure <code>useDiagramStore</code> selectors. Computes a health-dot color from validation issues.
AwsServiceNode	components/nodes/AwsServiceNode.tsx	<code>memo</code> -wrapped. Resolves an icon dynamically from lucide-react's full export map by string name. Right-click context menu (Duplicate/Copy ARN/Add note/Delete).
GroupBoxNode / LabelNode	components/nodes/	<code>memo</code> -wrapped. Double-click-to-rename inline editing pattern, each with its own <code>editing</code> state.
FlowEdge	components/edges/FlowEdge.tsx	Pure function of props — color-coded by connection type, animated flow dots for selected/event edges.

2.3 Dashboard pages

UPDATED IN REVISION 2

Five of the eleven pages below moved out of `DashboardShell.tsx` into their own files under `src/dashboard/pages/` (marked "Moved" below); the rest are still defined inline. Line numbers for the still-inline pages are approximate — they shift as the file continues to change; use `grep -n "^function PageName"` to find the current location rather than trusting a specific line number here.

Page	Location	What it does
OverviewPage	DashboardShell.tsx (inline)	Dashboard landing view — KPI grid, AWS graphs, resource/billing/events tables, cost recommendations, active-diagrams list. Purely presentational, no local state.
TerraformPage	<code>pages/TerraformPage.tsx</code> (Moved, 45 lines)	Static demo Terraform file/code viewer. No API calls.
AgentPage	<code>pages/AgentPage.tsx</code> (Moved, 141 lines)	AI "Well-Architected" chat UI. State: <code>conversation</code> , <code>draft</code> , <code>error</code> , <code>isSending</code> . Gated by <code>canUseAiAgent(user)</code> . Uses a one-off optimistic-update pattern (appends the user's message locally before the network call resolves).
DeploymentsPage	DashboardShell.tsx (inline)	Deployment table with destroy/force-destroy confirmation dialogs. Polls every 4s while any deployment is active. Renders <code>DeploymentTableDetails</code> for expandable rows.
ResourceInfoPage	DashboardShell.tsx (inline)	Deployment picker + <code>ResourceInfoViewer</code> . Seeds selected deployment from a <code>?deployment=</code> query param.
ApplicationPipelinePage	DashboardShell.tsx (inline)	Largest remaining inline page (~950 lines, 23 <code>useState</code> calls) . Full CI/CD builder: GitHub connect + repo/branch picker, build command config, Monaco-based generated-file viewer, GitHub Actions run-status polling (5s), OAuth popup handling via <code>postMessage</code> .
SupportPage	<code>pages/SupportPage.tsx</code> (Moved, 422 lines)	Ticket list + thread + reply form + new-ticket modal. 15 <code>useState</code> calls. Role-aware (super-admin sees a status-change control). Also picked up the file-size-limit and validation fixes from the Revision 2 changelog.
SuperAdminPage	<code>pages/SuperAdminPage.tsx</code> (Moved, 402 lines)	User table with role/status filters, per-user detail, credit-granting form, activity log. Hard-gated on <code>user?.role === 'superadmin'</code> .
SecurityPage	DashboardShell.tsx (inline)	Security findings list; falls back to static demo data when there's no live AWS insight data. Confirmed still live and linked (not dead code) during the Revision 2 pass.

ConnectAwsPage	pages/ConnectAwsPage.tsx (Moved, 170 lines)	AWS IAM role connection form + connected-accounts list.
----------------	--	---

REMOVED IN REVISION 2, NO LONGER IN THE CODEBASE

RuntimeLabPage — the Node.js-internals teaching demo (worker threads / child processes / cluster concepts) — was deleted outright, not extracted. It was already unlinked (sidebar entry commented out, reachable only by direct URL) and a full-codebase grep after removal found zero remaining references.

Supporting/leaf components still defined in `DashboardShell.tsx`: `KpiGrid`, `ResourceTable`, `PermissionErrorList`, `BillingServiceTable`, `RecentAwsEvents`, `DeploymentTableDetails`, `DeploymentDetailSection`, `DeploymentKeyValueTable`, `CostRecommendationGrid`, and `RuntimeLabDetailModal` (kept — despite its name, it's still used independently by `KpiGrid` and `ResourceTable` for their own detail popups, each with its own local `detail` state). `TicketAttachmentList` moved into `pages/SupportPage.tsx` with the rest of the ticket UI. `Panel` and `EmptyState` — the two most widely reused primitives — moved to `dashboard/components/DashPrimitives.tsx` specifically so the extracted page files could import them without creating a circular import back into `DashboardShell.tsx`.

2.4 Auth, settings, landing, reference

Component	File	Notes
AuthPage	auth/AuthPage.tsx (294 lines)	Props: <code>mode: 'login' 'register'</code> , theme. One flat component, no sub-decomposition — 13 <code>useState</code> calls covering all four flows (login/register/forgot/reset).
GithubSettingsPage	settings/GithubSettingsPage.tsx (162 lines)	Reads a <code>?github=connected error</code> redirect-notice param and strips it via <code>history.replaceState</code> .
LandingPage	landing/LandingPage.tsx (823 lines)	One shared piece of state (<code>detail</code>) threaded through ~12 static sub-sections via an <code>onOpenDetail</code> callback, driving one <code>LearningDetailModal</code> . Includes a hand-built draggable fake-diagram mockup with its own pointer-event state.
ReferenceDocsPage	reference/ReferenceDocsPage.tsx (127 lines)	Renders a static two-item doc list; selected doc shown in an <code><iframe></code> pointed at a backend-served PDF URL.

2.5 Custom hooks

An exhaustive search for `function use[A-Z]` patterns across the whole `src/` tree found **exactly one** hand-written custom hook in the entire application:

USESCROLLHINT

`dashboard/DashboardShell.tsx` (still inline; line number shifts as the file changes, was ~3134 as of Revision 2) — returns `{ showScrollHint }`. Locates the actual scrollable element (window, or the nearest `.dash-content` descendant with overflow), attaches scroll/resize listeners plus a `ResizeObserver`, and flags when there's more than 16px of unscrolled content. Its only consumer is `DashboardShell` itself, driving the `ScrollHintIcon`.

`useDiagramStore` is Zustand-generated, not a hand-written hook — see Chapter 3.

2.6 Error boundaries

RESOLVED IN REVISION 2

The original edition of this document found zero error boundaries anywhere — any unhandled render-time exception white-screened the whole app. `src/components/ErrorBoundary.tsx` (a class component; error boundaries can't be written as hooks) now wraps `<ReactFlowProvider><App></ReactFlowProvider>` in `main.tsx`. It's a single top-level catch-all, not per-page — a crash anywhere still takes down the whole dashboard's content area (replaced by a "something went wrong" card with a reload button), it just no longer white-screens with zero explanation. Verified with a real crash during the Revision 2 smoke test (a test-mock shape mismatch in

SuperAdminPage, not a code bug): the boundary caught it cleanly and logged the full component stack via `componentDidCatch`. See Chapter 10 for what to do when this fires.

State Management

3.1 The one Zustand store

`src/store/diagramStore.ts` (508 lines) is created with a plain `create<DiagramStore>((set, get) => ({...}))` call — **no middleware** (no `persist`, no `devtools`, no `immer`) — still true as of Revision 2. This means the entire diagram (nodes, edges, undo history) is still lost on a confirmed refresh/navigation unless it was explicitly saved to the backend via `diagramApi.ts` first. What Revision 2 added is a warning, not persistence: a new `isDirty` flag plus a `beforeunload` handler in `VisualBuilderPage` now prompts the browser's native confirmation dialog before an accidental refresh/close discards unsaved work — see Chapter 9, finding #5.

State shape

```
nodes: AwsNode[]
edges: AwsEdge[]
selectedNodeId?: string
selectedEdgeId?: string
inspectorNodeId?: string
inspectorEdgeId?: string
focusNodeIds: string[]
fitViewVersion: number // bumped to trigger react-flow's fitView() re-runs
mode: ToolMode
activeView: DiagramViewMode // default 'topology'
activeRegion: string // default 'ap-south-1'
lastSavedAt?: string
isDirty: boolean // NEW in Revision 2 – true since the last save/load; drives the beforeunload guard
isDark: boolean
issues: ValidationIssue[]
history: DiagramSnapshot[] // undo stack, capped at 50
future: DiagramSnapshot[] // redo stack
clipboard?: DiagramSnapshot // in-memory only, not persisted
```

Key actions

- **Node/edge CRUD:** `addServiceNode`, `addGroupNode`, `addLabelNode`, `onNodesChange` / `onEdgesChange` (wrap React Flow's `applyNodeChanges` / `applyEdgeChanges`), `onConnect`, `onEdgeUpdate`, `updateNodeData`, `updateNodeConfig`, binding CRUD, `duplicateSelection`, `deleteSelection`, `copySelection` / `pasteClipboard`, `selectAll`.
- **History:** every mutating action calls `pushHistory` first (truncates to last 50, clears the redo stack — a standard linear undo/redo, not a tree). `undo` / `redo` swap nodes/edges between the two stacks.
- **Validation:** `validate()` runs `validateDiagram(nodes, edges)` and stamps a warning message onto each affected node's data.
- **Persistence bridge:** `importDiagram(snapshot)` replaces nodes/edges wholesale (used when loading a saved/demo/template diagram); `markSaved()` just stamps a local timestamp string — it does *not* call any API itself (the actual save API call happens in `VisualBuilderPage`, which then calls `markSaved()`).

Subscription patterns — two styles, used inconsistently

`useDiagramStore` is imported in 10 files. Two different subscription styles coexist:

- **Whole-store destructure** (re-renders on *any* state change): `Canvas.tsx`, `PropertiesPanel.tsx`, `VisualBuilderPage`, `Toolbar.tsx`.
- **Selector-based** (subscribes to one field, minimizes re-renders): `StatusBar.tsx` (five separate single-field hooks), `Sidebar.tsx` (`state.nodes` only), the node components (single-action selectors like `state.updateNodeData`).

WORTH KNOWING WHEN DEBUGGING PERF

`Canvas.tsx` — the component rendering the potentially-large React Flow graph — subscribes to the *whole* store rather than selecting only what it needs. Any store mutation anywhere in the app (even one unrelated to nodes/edges, like `setDark`) will re-render it.

3.2 Everything else is local component state

There is **no React Context anywhere** in the codebase — a grep for `createContext` / `useContext` across all of `src/` returned zero matches. Theme (`ThemeMode`) and the current auth user are not shared via context; they are threaded as plain props from `App.tsx` downward (theme) or independently re-read from `localStorage` via the synchronous `getStoredUser()` helper in nearly every gated component (auth user). `diagramStore.ts` is confirmed to be the *only* Zustand store in the app (a grep for `from 'zustand'` matched only that one file).

Every dashboard page's own data (deployments, AWS insights, tickets, pipelines, notifications, etc.) lives in plain per-component `useState`, fetched fresh on mount — see Chapter 5 for the fetch pattern.

3.3 What's actually persisted client-side

Data	Where	Mechanism
Auth token + user	<code>localStorage['infra-auth-token']</code> / <code>['infra-auth-user']</code>	Plain key storage via <code>authClient.ts</code> , no expiry handling client-side beyond what the backend enforces.
Theme preference	<code>localStorage['infra-theme']</code>	Plain key storage, set in an <code>App.tsx</code> <code>useEffect</code> .
Diagram (nodes/edges/undo history)	<i>Nowhere</i> unless explicitly saved	Zustand store has no persist middleware — refresh loses everything not already saved via <code>diagramApi.ts</code> .

Routing

NO ROUTER LIBRARY

There is no react-router (or any routing library) in `package.json`. All routing is manual, built directly on `window.location`.

4.1 Top-level routes — App.tsx

`App.tsx` reads `path = window.location.pathname` once per render and picks a page via sequential `if` statements (not a switch — later matches override earlier ones in the variable assignment, functionally acting like a route table):

Path	Renders	Guarded?
<code>/</code> (and any unmatched path — no explicit 404)	<code>LandingPage</code>	No
<code>/dashboard</code>	<code>DashboardShell</code>	Yes
<code>/settings</code> , <code>/profile</code>	<code>GithubSettingsPage</code> (same component for both paths)	Yes
<code>/references</code>	<code>ReferenceDocsPage</code>	No
<code>/login</code>	<code>AuthPage</code> mode="login"	No
<code>/register</code>	<code>AuthPage</code> mode="register"	No

4.2 The auth guard

For the three protected paths, a `useEffect` (deps: `[path, requiresAuth]`) calls `validateStoredSession()` from `auth/authClient.ts`. While pending, the guarded route renders a "Checking dashboard/settings access..." placeholder instead of the real page.

```
validateStoredSession():
  1. Read token from localStorage — if absent, clear session and return null (no network call).
  2. GET {API_BASE_URL}/auth/me with Authorization: Bearer <token>
  3. Expect { success: true, data: { user } } — anything else clears the session and returns null.
  4. On success, re-store the token + fresh user object, return the user.

On null → App.tsx does window.location.replace('/login?next=' + encodeURIComponent(path+search))
          (a full page navigation, not a client-side transition)
```

4.3 Dashboard-internal "sub-routing" — the `?view=` pattern

`DashboardShell.tsx` implements its own second routing layer for the ~11 dashboard pages (12 in the original edition of this document — Runtime Lab has since been removed), entirely independent of any router library:

- `getInitialDashboardPage()` reads `?view=` from the URL via `URLSearchParams`, validates it against the known `DashboardPage` set, and falls back to `'overview'`. This seeds `activePage` state.
- `goToDashboardPage(page)` calls `setActivePage(page)` and, as of Revision 2, `window.history.pushState(null, '', getDashboardUrl(page))` (was `replaceState` — see below).
- `renderPage(activePage, ...)` is a plain switch function (not a component) mapping each `DashboardPage` value to its page component.

RESOLVED IN REVISION 2 — BROWSER BACK/FORWARD NOW WORKS

The original edition of this document flagged that `history.replaceState` meant switching dashboard views created no browser history entries — the back button skipped over all of them and left the dashboard entirely. Fixed by switching to `pushState` plus a new `popstate` listener (a `useEffect` in `DashboardShell` that calls `setActivePage(getInitialDashboardPage())` whenever the

browser fires a back/forward navigation). Verified live: Overview → Deployments → Security, then two backs and a forward, correctly stepped through each URL and kept the sidebar's active-item highlight in sync at every step. `goToResourceInfo` was switched the same way.

Two pages read an *additional* query param on top of `?view=`: `ResourceInfoPage` reads `?deployment=<id>` (written by `goToResourceInfo(deploymentId)`), and `VisualBuilderPage` reads `?updateDeployment=<id>` once on mount to preload a diagram for editing, then immediately strips it back to a clean `/dashboard?view=builder` URL.

4.4 Sidebar visibility vs. actual access control

SIDEBAR FILTERING IS NOT ACCESS ENFORCEMENT

`visibleNavItems` hides the AI Agent, Application Pipeline, and Super Admin entries from users who shouldn't see them (via `canUseAiAgent`, `canUseApplicationPipelines`, and a role check) — but this only filters the sidebar *list*. Nothing at the routing layer stops `renderPage` from rendering a hidden page if someone navigates to its `?view=` URL directly. Each gated page re-checks access itself internally (e.g. `AgentPage` shows a paywall instead of the chat UI if `canUseAiAgent(user)` is false) — so it isn't a real bypass, but the enforcement point is scattered per-page rather than centralized in the router.

API / Data Layer

5.1 How the frontend talks to the backend

Every network call is a raw `fetch()` — there is no axios, no React Query/SWR. As of Revision 2 there is a shared HTTP client module, `src/utils/apiClient.ts`, exporting `apiRequest` and `apiFormRequest` (for multipart uploads).

RESOLVED IN REVISION 2 — WAS: 12 FILES, 12 NEAR-IDENTICAL COPIES

The original edition of this document found every API file independently defining its own local request helper — near-identical copy-paste of the same `fetch/header/error-unwrap` logic. Most now delegate to the shared client via a thin per-file wrapper that preserves each file's own fallback error message, e.g.:

```
import { apiRequest as sharedApiRequest } from '../utils/apiClient';
const diagramRequest = <T,>(path: string, init?: RequestInit) =>
  sharedApiRequest<T>(path, init, 'Diagram request failed');
```

Three files were deliberately left *not* using the shared client, because their contracts are genuinely different, not just copy-paste:

`authClient.ts` (`validateStoredSession()` must return `null` instead of throwing — `App.tsx` 's auth guard needs a falsy sentinel, not a catchable exception), `ticketApi.ts` 's `fetchTicketAttachmentBlobUrl()` (reads a `Blob`, not JSON), and `terraformPayloadApi.ts` (returns the whole response envelope, not just its `data` field).

The shared shape (`utils/apiClient.ts`)

```
export const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ?? '/api/v1';

async function unwrap<T>(response: Response, fallbackMessage: string): Promise<T> {
  const result = await response.json().catch(() => null);
  if (!response.ok || !result?.success) {
    throw new Error(result?.message ?? fallbackMessage);
  }
  return result.data as T;
}

export async function apiRequest<T>(path: string, init: RequestInit = {}, fallbackMessage = 'Request failed'):
Promise<T> {
  const token = getStoredToken();
  const response = await fetch(`${API_BASE_URL}${path}`, {
    ...init,
    credentials: 'include',
    headers: { 'Content-Type': 'application/json', ...(token ? { Authorization: `Bearer ${token}` } : {}),
    ...init.headers },
  });
  return unwrap<T>(response, fallbackMessage);
}
```

RESOLVED IN REVISION 2 — AGENTAPI.TS WAS AN ACTUAL BUG, NOT JUST A RISK

The original edition flagged `dashboard/agentApi.ts` for using a different env var and a hardcoded localhost fallback (`http://127.0.0.1:4001/api/v1`), framed as "this implies a separate service and will break in deployed environments." Investigation during the Revision 2 pass found it was worse than a risky fallback — it was simply wrong. `IAAS backend/src/routes/index.js` mounts `apiRouter.use('/agent', agentRouter)` on the exact same Express app as every other route (`/deployments`, `/tickets`, etc.); that router internally proxies to a separate RAG service via `RAG_API_URL` — but that proxying happens *server-side*, invisible to the frontend. There was never a real reason for the frontend to target a different port. `agentApi.ts` now uses the same shared client and base URL as everything else.

5.2 Full endpoint catalog

auth/authClient.ts

Method	Function	Endpoint	Returns
POST	login(payload)	/auth/login	{ accessToken, user }
POST	register(payload)	/auth/register	{ accessToken, user }
POST	forgotPassword({email})	/auth/forgot-password	ForgotPasswordResponse
POST	resetPassword({token,password})	/auth/reset-password	ResetPasswordResponse
POST	logout()	/auth/logout	best-effort; always clears local session
GET	validateStoredSession()	/auth/me	AuthUser null (never throws)

utils/deploymentApi.ts

Method	Function	Endpoint	Returns
POST	createCanvasDeployment(payload)	/deployments/from-canvas	DeploymentRecord
POST	uploadLambdaZip(file)	/deployments/lambda-zip (multipart)	{ uploadId, fileName, sizeBytes }
GET	getDeployment(id)	/deployments/:id	DeploymentRecord
POST	updateDeployment(id, payload)	/deployments/:id/update	DeploymentRecord
GET	listDeployments()	/deployments	DeploymentRecord[]
POST	applyDeployment(id)	/deployments/:id/apply	DeploymentRecord <i>(exported, no call site found in src/ — appears unused)</i>
POST	destroyDeployment(id)	/deployments/:id/destroy	DeploymentRecord
POST	forceDestroyDeployment(id)	/deployments/:id/force-destroy	DeploymentRecord

utils/terraformPayloadApi.ts

Method	Function	Endpoint	Returns
POST	sendTerraformPayload({nodes,edges,activeRegion})	/terraform-payload	TerraformPayloadResponse

dashboard/agentApi.ts

Method	Function	Endpoint	Returns
POST	createAgentConversation(message?)	/agent/conversations	AgentConversation
POST	sendAgentMessage(id, message)	/agent/conversations/:id/messages	AgentConversation

dashboard/applicationPipelineApi.ts

Method	Function	Endpoint	Returns
GET	listApplicationPipelines()	/app-pipelines	ApplicationPipelineRecord[]
POST	createApplicationPipeline(payload)	/app-pipelines	ApplicationPipelineRecord
POST	syncPipelineToGithub(id, {...})	/app-pipelines/:id/github-sync	{ pipeline, sync, oidc }
POST	deployApplicationPipeline(id, {...})	/app-pipelines/:id/deploy	ApplicationDeploymentStatus
GET	getApplicationDeploymentStatus(id, {...})	/app-pipelines/:id/deployment-status	ApplicationDeploymentStatus

POST	reportPipelineRunResult(id, payload)	/app-pipelines/:id/run-result	{ recorded }
------	--------------------------------------	-------------------------------	--------------

dashboard/awsApi.ts

Method	Function	Endpoint	Returns
GET	listAwsRegions()	/aws/regions	string[]
GET	listAwsAccounts()	/aws/accounts	AwsAccountRecord[]
POST	connectAwsAccount(payload)	/aws/accounts	AwsAccountRecord
POST	syncAwsAccount(id)	/aws/accounts/:id/sync	AwsAccountRecord
DELETE	disconnectAwsAccount(id)	/aws/accounts/:id	AwsAccountRecord — called from <code>ConnectAwsPage</code> 's <code>handleDisconnect</code> (confirmed directly during the Revision 2 extraction; the original edition's "no call site found" was a research miss, not a real change)
GET	getAwsInsights()	/aws/insights	AwsInsights (large nested billing/security/inventory shape)
GET	getDeployerIdentity()	/aws/deployer-identity	{ arn, accountId } <i>(no call site found — appears unused)</i>
GET	listAccountIamRoles(accountId)	/aws/accounts/:accountId/iam-roles	IamRoleSummary[]

dashboard/diagramApi.ts

Method	Function	Endpoint	Returns
GET	listSavedDiagrams()	/diagrams	SavedDiagram[]
POST	createSavedDiagram(payload)	/diagrams	SavedDiagram
PATCH	updateSavedDiagram(id, payload)	/diagrams/:id	SavedDiagram
DELETE	deleteSavedDiagram(id)	/diagrams/:id	{ message? }

REMOVED IN REVISION 2

`dashboard/nodeLabApi.ts` (`getNodeRuntimeSnapshot()` → `GET /node-lab/snapshot` , `runNodeConceptDemo()` → `POST /node-lab/run`) no longer exists — deleted along with the rest of the Runtime Lab feature. The backend routes it called (`/node-lab/*`) were removed too.

dashboard/notificationApi.ts

Method	Function	Endpoint	Returns
GET	listNotifications()	/notifications	{ notifications, unreadCount }
POST	markNotificationRead(id)	/notifications/:id/read	NotificationRecord <i>(no call site found — appears unused)</i>
POST	markAllNotificationsRead()	/notifications/read-all	{ updated }

dashboard/superAdminApi.ts

Method	Function	Endpoint	Returns
GET	getSuperAdminOverview()	/superadmin/overview	SuperAdminOverview
PATCH	updateSuperAdminUserRole(id, role)	/superadmin/users/:id/role	{ id, role }
POST	grantSuperAdminCredits(id, credits, note?)	/superadmin/users/:id/credits	{ id, demoCredits }
POST	requestDemoCredits(requestedCredits, reason)	/superadmin/credits/request	{ creditRequest }

dashboard/ticketApi.ts

Method	Function	Endpoint	Returns
GET	listTickets(status?)	/tickets?status=	TicketSummary[]
GET	getTicket(id)	/tickets/:id	TicketDetail
POST	createTicket({...files})	/tickets (multipart)	TicketDetail
POST	addTicketComment(id,{...files})	/tickets/:id/comments (multipart)	TicketDetail
PATCH	updateTicketStatus(id, status)	/tickets/:id/status	TicketDetail
GET	fetchTicketAttachmentBlobUrl(attachment)	<attachment.url>	blob: URL (raw fetch, bypasses the shared helper — see 5.3)

github/githubApi.ts

Method	Function	Endpoint	Returns
—	githubOAuthUrl({mode,returnTo})	/github/oauth/connect?... (builds a URL, doesn't fetch)	string, used for window.location.href redirect
GET	getGithubStatus()	/github/status	GithubConnection
DELETE	disconnectGithub()	/github/disconnect	GithubConnection
GET	listGithubRepositories()	/github/repos	GithubRepository[]
GET	listGithubBranches(owner, repo)	/github/branches?owner=&repo=	GithubBranch[]

`githubApi.ts` has a quirk not present anywhere else: it cache-busts GET requests by appending `?_=<timestamp>` and forces `cache: 'no-store'` plus a `Cache-Control: no-cache` header.

5.3 Error handling — one deliberate exception

The shared pattern throws an `Error` on any non-success response. `validateStoredSession()` is the one function in the whole codebase that deliberately does *not* throw — it swallows failure and returns `null`, because `App.tsx`'s auth guard needs a falsy sentinel to redirect on, not a catchable exception.

`fetchTicketAttachmentBlobUrl` also bypasses the shared `ticketRequest` helper entirely — it needs the raw `Response` to call `.blob()` on, with a code comment explaining why: attachment URLs need the bearer token attached via a fetch header rather than a query string, to avoid the token leaking into browser history/Referer headers the way a plain `<img src>` would.

5.4 Loading, caching, and retry — or the absence of them

- **No shared data-fetching hook exists.** Every component hand-rolls its own `isLoading` / `error` / `data` `useState` trio and its own `async function refreshX()` wrapped in `try/catch/finally`, called from a `mount` `useEffect` and again from manual "Refresh" buttons. This exact idiom repeats independently in nearly every dashboard page.
- **No retry logic anywhere.** A grep for "retry" across `src/` found only user-facing copy telling a human to retry manually — no exponential backoff, no automatic re-attempt, no `AbortController`-based cancel-and-retry.
- **No client-side caching.** Nothing fetched from the API is written into the Zustand store or any browser storage with invalidation logic — every remount/navigation refetches from scratch.
- **Polling stands in for real-time updates**, via plain `window.setInterval`:
 - `DashboardShell.tsx` polls `listNotifications()` every 15,000ms unconditionally while mounted.
 - `DeploymentsPage` polls `listDeployments()` every 4,000ms, but only while at least one deployment is `queued` / `deploying` / `destroying`.
 - `DeploymentModal` polls deployment status every 2,200ms while a deploy is running.
 - `ApplicationPipelinePage` polls GitHub Actions run status every 5,000ms.

All are cleared in their effect's cleanup function.

Forms & Validation

NO FORM LIBRARY

No react-hook-form, formik, or yup anywhere in the codebase. Every form is plain `useState` per field plus hand-written conditional checks.

6.1 AuthPage — login / register / forgot / reset

`src/auth/AuthPage.tsx`. Validation is computed as plain booleans (`canSubmit`, `canRequestPasswordReset`, `canResetPassword`) that gate the submit button's `disabled` state — there is no per-field inline error UI, only one shared error slot populated from a caught API exception.

Flow	Rule
Login	email non-empty (no format regex — relies on the HTML <code>type="email"</code> attribute only); password length ≥ 8 .
Register	same as login, plus <code>name.trim().length ≥ 2</code> .
Forgot password	email non-empty.
Reset password	<code>resetToken.trim().length ≥ 32</code> ; new password length ≥ 8 .

6.2 Connect AWS account form

`ConnectAwsPage` — `src/dashboard/pages/ConnectAwsPage.tsx` as of Revision 2 (previously inline in `DashboardShell.tsx`). Fields: account name, AWS account ID, IAM role ARN, external ID (optional), default region. Validation is purely native HTML5 `required` attributes on name/accountId/roleArn — no client-side format checking of the account ID digits or ARN shape (presumably left to the backend). Errors shown via one flat success/error message pair, not per-field.

6.3 PropertiesPanel — the resource-config form

This is the most sophisticated form in the app, and the only one with genuine per-field inline error rendering.

Field type system (`AwsField.type`, `types.ts`)

```
'text' | 'select' | 'number' | 'json' | 'iam-role' | 'file-path'
```

- select** — native `<select>` from `field.options`.
- json** — opens `JsonFieldModal`; a textarea that attempts `JSON.parse` and shows "Not valid JSON: ..." but **still allows saving invalid JSON** (deliberate — Terraform expressions like `jsonencode(...)` are also legal values there).
- iam-role** — opens `IamRolePickerModal`: browse real IAM roles fetched live from the connected AWS account, paste a custom ARN, or author a brand-new role (JSON policy or raw Terraform snippet).
- file-path** — `FilePathField`: text input + native file picker that uploads a `.zip` via `uploadLambdaZip` and swaps in the returned `uploadId` (browsers never expose real filesystem paths, so this uploads bytes instead of trusting a typed path).

Required-field logic

REQUIRED-NESS IS COMPUTED, NOT DECLARED

A grep of `src/data/awsServices.ts` found **zero** occurrences of `required: true` on any field definition. In practice, required-ness flows entirely from `resourceRequirements.ts`'s hand-maintained `serviceRequirements` map (per-service required/recommended/sensitive/conditional lists for ~40 AWS services), combined with `isResolvableViaConnection` logic that treats a field as

satisfied if it's wired to the right kind of connected node (e.g. a subnet's `vpc_id` counts as filled if connected to a VPC node). If you're debugging "why does this field say required when I set it" or vice versa, **look in `resourceRequirements.ts`, not in the field's own definition in `awsServices.ts`.**

Errors render inline via the shared `Field` component: a `field--error` class plus a "Required before deployment" message under the specific input.

6.4 Ticket creation form

`SupportPage` — `src/dashboard/pages/SupportPage.tsx` as of Revision 2 (previously inline in `DashboardShell.tsx`). Fields: subject, category (select), priority (select), description (textarea), attachments (multi-file).

RESOLVED IN REVISION 2 — WAS: STATED LIMIT NOT ENFORCED, INCONSISTENT UX

The original edition of this document found the attachment field's "max 10MB each" hint text was not actually enforced anywhere client-side, and that the form's validation UX (no `required` attributes, submit-time-only checking, button not preemptively disabled) was inconsistent with `AuthPage/ConnectAwsPage`'s disabled-until-valid pattern. Both fixed: subject/description now have `required`, the submit button is disabled until `canSubmitNewTicket` (both fields non-empty) is true, and a new `selectTicketFiles()` helper rejects any file over `TICKET_MAX_ATTACHMENT_BYTES` (10MB, mirroring the backend's actual limit) at selection time on both the new-ticket and reply file pickers, naming the specific file(s) rejected.

6.5 validate.ts is not a form validator

`src/utils/validate.ts` (83 lines) is a diagram/architecture linter, imported by the visual builder — not related to any of the forms above. It exports `validateDiagram(nodes, edges): ValidationIssue[]`, layering architectural warnings (e.g. "RDS should be placed in a private subnet", "Sensitive binding appears to have a raw value in node config" via a `/password|secret|token|key/i` regex check) on top of the required-field checks from `resourceRequirements.ts`.

Styling Approach

RESOLVED IN REVISION 2 — TAILWIND REMOVED, NOT JUST UNUSED

The original edition of this document found Tailwind fully wired into the build (`tailwind.config.js` , `@tailwind` base/components/utilities at the top of `styles.css`) with zero actual usage — an exhaustive grep of every `.tsx` file for utility-class patterns turned up only false positives (semantic names like `"lp-card-grid"` that happen to contain the substring "grid"). Since nothing used it, it was removed outright rather than left as dead weight: `tailwind.config.js` and `postcss.config.js` deleted, the three `@tailwind` directives dropped from `styles.css` , and `tailwindcss` / `postcss` / `autoprefixer` uninstalled (63 packages removed from `node_modules`). Verified with a full `tsc && vite build` afterward — no PostCSS config is needed at all now, since `styles.css` is plain CSS.

7.1 The real stylesheet

`src/styles.css` is **14,601 lines** and is the entire visual layer of the application: hand-written, semantic, feature-prefixed class names (see Chapter 8). No CSS Modules (`.module.css`), no styled-components, no Emotion — all confirmed absent.

7.2 Theming — dark / light / pearl

`src/theme.ts` (23 lines) exports the `ThemeMode = 'dark' | 'light' | 'pearl'` type, a `themeModes` array, an `isThemeMode()` type guard, and `getNextTheme()` which cycles dark → light → pearl → dark. It's pure logic; the actual DOM class application happens in `App.tsx` :

```
theme === 'pearl'
  ? 'app-theme app-theme-light app-theme-pearl'
  : `app-theme app-theme-${theme}` + (theme === 'dark' ? 'dark' : '')
```

PEARL MODE STACKS ON TOP OF LIGHT MODE

Pearl isn't a fully independent third theme — it applies *both* `app-theme-light` and `app-theme-pearl` classes simultaneously, so pearl is really "light mode, plus ~450 additional override rules." If a color looks wrong in pearl mode but correct in light mode, check whether a `.app-theme-pearl` -scoped rule is present for that element — the `.app-theme-light` rule alone won't be overridden without one.

Confirmed by sampling the stylesheet: unscoped rules (no theme-class prefix) are the **dark-mode default** (e.g. `.dash-page { ... }` with no prefix). `.app-theme-light` (773 occurrences) and `.app-theme-pearl` (453 occurrences) prefix the override rules. The primary mechanism is CSS custom-property redefinition — `.app-theme-light` resets design tokens like `--bg-app` , `--bg-surface` , `--text-primary` , `--accent-primary` — layered with roughly 1,226 more specific component-level selector overrides for cases a variable swap alone doesn't cover.

Known Patterns & Conventions

8.1 Naming

What	Convention
Components	PascalCase, consistently — including trivial one-line wrapper components (<code>Code2Icon</code> , <code>LifeBuoyIcon</code>).
Component files	PascalCase (<code>Canvas.tsx</code>).
Non-component files	camelCase (<code>diagramStore.ts</code> , <code>awsApi.ts</code>).
Hooks	<code>use</code> + PascalCase — standard, though only one hand-written hook exists (<code>useScrollHint</code>).
Boolean props	Consistently prefixed <code>is</code> / <code>can</code> / <code>has</code> / <code>show</code> (<code>isCollapsed</code> , <code>canSaveDiagram</code> , <code>hasOpenableDiagrams</code> , <code>showDeploymentSuccess</code>).
Callback props	Consistently prefixed <code>on</code> (<code>onToggleTheme</code> , <code>onSaveDiagram</code> , <code>onViewResourceInfo</code>).
CSS classes	Hand-written, semantic, feature-scoped short prefixes rather than one global prefix: <code>dash-*</code> (dashboard shell), <code>lp-*</code> (landing page), <code>pipeline-*</code> (CI/CD builder), <code>ticket-*</code> , <code>admin-*</code> , <code>runtime-lab-*</code> , <code>resource-info-*</code> , <code>auth-*</code> , <code>settings-*</code> , <code>reference-*</code> . BEM-influenced modifiers via double-dash (<code>dash-page--builder</code>) and occasional double-underscore for sub-parts (<code>modal__header</code>). Status/variant suffixes are frequently template-literal-interpolated directly from data, e.g. <code>`status-pill status-pill--\${status}`</code> — this exact pattern recurs across nearly every page.

8.2 Prop drilling — mostly avoided, not a real problem

The visual builder side (`Canvas` , `Sidebar` , `Toolbar` , `PropertiesPanel` , `StatusBar` , node components) all read the Zustand store directly rather than receiving diagram state as props. On the dashboard side, page-level state lives in `DashboardShell` and is handed down exactly one level via the `renderPage()` switch. Nothing observed during this investigation reaches three or more untouched prop layers.

8.3 The one custom hook, and the absence of error boundaries

Covered in depth in Chapter 2 (§2.5–2.6): `useScrollHint` is the only hand-written hook in the codebase; there are zero error boundaries anywhere.

8.4 Console output

A grep for `console.` across all of `src/` found essentially nothing left over from debugging: one genuine call (`console.info(result.message)` in `Toolbar.tsx:164`) and one false positive (the word "console" inside a UI string in `DashboardShell.tsx`). This is a notably clean codebase on this specific axis.

Identified Risks / Technical Debt

This was originally a watch list only — nothing fixed, only recorded. As of **Revision 2**, 7 of these 10 numbered findings have been resolved (2 fully as originally scoped, plus one item turned out to be a real bug rather than just a risk). See the changelog after the table of contents for the full list with what specifically changed; each item below is updated in place with its current status.

1. DASHBOARD.SHELL.TSX MONOLITH — PARTIALLY RESOLVED

Originally: 4,985 lines, 85 top-level functions, simultaneously the dashboard's page router, ~20 page-level components, and dozens of business-logic helpers for roughly ten unrelated product areas. As of Revision 2: 5 self-contained pages (AgentPage, ConnectAwsPage, SupportPage, SuperAdminPage, TerraformPage) were extracted to `src/dashboard/pages/`, plus a shared `DashPrimitives.tsx` for `Panel / EmptyState` — **3,439 lines now**, a 31% reduction. The "Node.js runtime lab" teaching demo mentioned in the original finding no longer exists at all (deleted). What's *not* resolved: Overview + its KPI/resource widgets, the visual builder host, Deployments + its large metrics subsystem, the ~950-line Application Pipeline builder, and Security are all still inline and still interdependent enough that extracting them was deliberately left for a follow-up pass rather than rushed. This remains the highest-leverage refactor candidate.

2. ZERO ERROR BOUNDARIES — RESOLVED

`src/components/ErrorBoundary.tsx` now wraps `<ReactFlowProvider><App></ReactFlowProvider>` in `main.tsx`. It's one top-level boundary, not per-page, so a crash still takes down the dashboard's content area — it just shows a "something went wrong" card with a reload button instead of a blank white screen. Verified with a real crash during the Revision 2 smoke test (a test-mock shape mismatch, not an app bug): caught cleanly, full stack logged via `componentDidCatch`. See Chapter 10.

3. ~10 NEAR-IDENTICAL API REQUEST HELPERS — RESOLVED

New shared `src/utils/apiClient.ts` (Chapter 5, §5.1). Nine files now delegate to it via a thin per-file wrapper that preserves their own fallback error message. `authClient.ts`, `ticketApi.ts`'s blob fetch, and `terraformPayloadApi.ts` deliberately still don't, since their contracts are genuinely different (see Chapter 5 for why). Centralized changes like 401 handling or request logging can now be made in one place for the nine that do.

4. AGENTAPI.TS HARDCODED LOCALHOST FALLBACK — RESOLVED (TURNED OUT TO BE A REAL BUG)

Investigation found this wasn't just a risky fallback — `/agent/*` is served by the exact same backend Express app as every other route (`apiRouter.use('/agent', agentRouter)` in `routes/index.js`), which itself proxies server-side to a separate RAG service. There was never a real separate frontend-facing service on port 4001. `agentApi.ts` now uses the shared client and the same relative `/api/v1` base URL as everything else.

5. NO DIAGRAM AUTOSAVE — PARTIALLY RESOLVED

The Zustand store still has no persistence middleware — a confirmed navigation away still discards everything. What changed: a new `isDirty` flag (set by `pushHistory`, cleared by `markSaved / importDiagram`) plus a `beforeunload` handler in `VisualBuilderPage` now prompts the browser's native "leave site?" dialog if there are unsaved changes when the user tries to refresh or close the tab. This is a warning, not autosave-to-server or autosave-to-localStorage — full loss-prevention would need one of those and wasn't attempted here.

6. DASHBOARD NAVIGATION DOESN'T INTEGRATE WITH BROWSER HISTORY — RESOLVED

`goToDashboardPage / goToResourceInfo` switched from `history.replaceState` to `pushState`, plus a new `popstate` listener that re-syncs `activePage` on back/forward. Verified live: Overview → Deployments → Security, two backs, one forward — each step correctly changed the URL, the rendered page, and the sidebar's active-item highlight.

7. INCONSISTENT TICKET FORM VALIDATION — RESOLVED

The new-ticket submit button is now disabled until subject+description are filled, matching AuthPage/ConnectAwsPage's pattern. The stated 10MB-per-file attachment limit (now exported as `TICKET_MAX_ATTACHMENT_BYTES` from `ticketApi.ts`, mirroring the backend's own limit) is enforced on file selection for both the new-ticket and reply pickers, with a specific error naming which file(s) were rejected and why.

8. REQUIRED-NESS FOR RESOURCE CONFIG FIELDS IS COMPUTED, NOT DECLARED ON THE FIELD

`awsServices.ts` never sets `field.required` — it's entirely driven by a separate hand-maintained map in `resourceRequirements.ts` (Chapter 6, §6.3). Anyone adding a new field to a service definition and expecting to mark it required by setting a property directly on the field object will find it has no effect; the requirement has to be added in the other file.

9. THE JSON CONFIG-FIELD EDITOR SILENTLY ALLOWS SAVING INVALID JSON

By design, since Terraform expression syntax (`jsonencode(...)`, etc.) is also a legal value for these fields — but this means a genuine typo/syntax error in a JSON policy field will not be caught by the UI at all, only surfaced later at Terraform-generation or deploy time.

10. TAILWIND FULLY WIRED BUT UNUSED — RESOLVED

Removed rather than left as dead weight: `tailwind.config.js` and `postcss.config.js` deleted, the `@tailwind` directives dropped from `styles.css`, and `tailwindcss` / `postcss` / `autoprefixer` uninstalled (63 packages). Verified with a full production build afterward. See Chapter 7.

11. PAST INCIDENT, NOW FIXED — MODULE-LOAD-TIME FILESYSTEM WRITES

Not currently a live bug (patched in a prior session covered by this same investigation's git history), but worth knowing the pattern exists: two backend upload middlewares (`ticketUpload.js`, `lambdaZipUpload.js`) originally called `mkdirSync()` at module-load time against a path inside their own deployed source tree. This is safe on a normal server but would crash *every* route on cold start if that backend were ever deployed somewhere with a read-only filesystem (e.g. AWS Lambda). Both were patched with a try/catch guard. If a new upload-handling middleware is added following the same copy-paste pattern without the guard, the same class of bug will recur.

12. PAST INCIDENT, NOW FIXED — LAMBDA DEPLOYMENT PACKAGE REFERENCED A BROWSER-LOCAL PATH

The Lambda node's "Deployment zip path" field originally stored only the picked file's *name* (browsers cannot expose a real filesystem path), which got embedded verbatim into generated Terraform as the `filename` argument — a path that could never resolve on the backend server actually running `terraform apply`. Fixed by making the field genuinely upload the file's bytes to the backend and reference the server-stored copy. Documented here as an example of the class of bug worth watching for elsewhere: any field in `PropertiesPanel.tsx` that *looks like* it's capturing a real filesystem/local value from the browser should be checked for whether it actually transfers bytes, or just a name/label.

Debugging Quick-Reference

Symptom → the files most likely to be load-bearing for it, based on the architecture documented in the preceding chapters.

Symptom	Check first
Login/register form's submit button stays disabled	<code>src/auth/AuthPage.tsx</code> — <code>canSubmit</code> / <code>canRequestPasswordReset</code> / <code>canResetPassword</code> booleans (Ch. 6.1). Remember: no inline per-field errors exist here, only the disabled state itself.
User is redirected to <code>/login</code> unexpectedly from <code>/dashboard</code>	<code>src/App.tsx</code> <code>auth-guard</code> <code>useEffect</code> ; <code>src/auth/authClient.ts</code> <code>validateStoredSession()</code> ; check <code>localStorage['infra-auth-token']</code> and confirm <code>GET /auth/me</code> is actually returning <code>{success:true,data:{user}}</code> (Ch. 4.2).
A dashboard page shows stale data after an action elsewhere	There is no shared cache (Ch. 5.4) — every page refetches independently. Find that page's <code>refreshX()</code> function inside <code>DashboardShell.tsx</code> and check its <code>useEffect</code> dependency array; if the triggering action doesn't call the same refresh function or bump a dependency, the page simply won't know to refetch.
Visual builder canvas doesn't reflect a state change	<code>src/store/diagramStore.ts</code> — confirm the action exists and calls <code>set(...)</code> correctly; then check whether the consuming component uses a whole-store destructure or a selector (Ch. 3.1) — a selector-based component (e.g. <code>StatusBar</code> , <code>Sidebar</code>) will not re-render for fields it didn't select.
A node's Properties Panel shows the wrong fields, or a field wrongly marked required	<code>src/data/awsServices.ts</code> for the field's existence/type; <code>src/utils/resourceRequirements.ts</code> for its actual required/recommended/conditional status — not the field definition itself (Ch. 6.3, risk #8).
Lambda deployment fails with a "file not found" / zip error	Backend <code>terraformGenerator.js</code> 's <code>case 'lambda'</code> and <code>lambdaDeploymentPackageFromField</code> ; <code>terraformDeploymentRunner.js</code> 's <code>placeLambdaZipUploads</code> ; confirm the uploaded zip actually exists under <code>IAAS backend/uploads/lambda-zips/</code> . See risk #12 for the historical version of this bug.
IAM role picker is empty, or "create new role" doesn't work	<code>src/components/PropertiesPanel.tsx</code> 's <code>IamRolePickerModal</code> ; <code>src/dashboard/awsApi.ts</code> <code>listAccountIamRoles</code> ; confirm an AWS account is actually connected (Ch. 6.3) and the backend <code>/aws/accounts/:id/iam-roles</code> call is succeeding.
Colors/contrast look wrong specifically in light or pearl mode	<code>src/App.tsx</code> 's theme-class assembly (pearl stacks <i>both</i> <code>app-theme-light</code> and <code>app-theme-pearl</code> — Ch. 7.2); then search <code>src/styles.css</code> for the element's class under both the <code>.app-theme-light</code> and <code>.app-theme-pearl</code> scopes — a missing pearl-scoped override is a common cause.
AI Agent page shows a paywall for a user who should have access	<code>src/utils/accessControl.ts</code> — <code>canUseAiAgent()</code> , called from <code>src/dashboard/pages/AgentPage.tsx</code> (moved here in Revision 2). <code>dashboard/agentApi.ts</code> 's base-URL confusion (old Ch. 9 risk #4) is fixed — no longer a separate env var to check.
Agent chat requests fail even though the user has access	<code>src/dashboard/agentApi.ts</code> now shares <code>API_BASE_URL</code> / <code>apiClient.ts</code> with everything else — if this specifically breaks, suspect the backend's <code>/agent</code> route or its <code>RAG_API_URL</code> proxy target, not a frontend base-URL mismatch.
GitHub OAuth "connect" popup never completes / hangs	<code>src/settings/GithubSettingsPage.tsx</code> and <code>ApplicationPipelinePage</code> 's <code>postMessage</code> listener + <code>githubPopupRef</code> / <code>githubPollRef</code> ; <code>src/github/githubApi.ts</code> <code>githubOAuthUrl()</code> .
Notification bell count is wrong or doesn't update	<code>DashboardShell.tsx</code> 's 15-second notification-polling <code>useEffect</code> ; <code>src/dashboard/notificationApi.ts</code> .
Ticket attachments fail to display	<code>src/dashboard/ticketApi.ts</code> <code>fetchTicketAttachmentBlobUrl</code> (bypasses the normal request helper on purpose — Ch. 5.3) and the backend's ticket-upload middleware/controller. UI now lives in <code>src/dashboard/pages/SupportPage.tsx</code> .
New-ticket submit button won't enable, or a large attachment silently doesn't attach	<code>src/dashboard/pages/SupportPage.tsx</code> — <code>canSubmitNewTicket</code> and <code>selectTicketFiles</code> (checks <code>TICKET_MAX_ATTACHMENT_BYTES</code> from <code>ticketApi.ts</code>). Fixed in Revision 2 — previously unenforced client-side.
The app white-screens, or shows a "something went wrong" card	As of Revision 2 there's a top-level <code>src/components/ErrorBoundary.tsx</code> — a real crash now shows a fallback card with a reload button instead of a blank screen. Either way, open the browser console for the actual stack trace (the boundary logs it via <code>componentDidCatch</code>). Given the size of <code>DashboardShell.tsx</code> , bisect via the active <code>?view=</code> value — check <code>src/dashboard/pages/</code> first if the crashing page is <code>Agent/ConnectAws/Support/SuperAdmin/Terraform</code> , otherwise it's still inline in <code>DashboardShell.tsx</code> .

Dashboard back/forward button seems to do nothing, or jumps to the wrong view	<code>DashboardShell.tsx</code> 's <code>popstate</code> listener and <code>getInitialDashboardPage()</code> (Ch. 4.3, fixed in Revision 2 — was previously non-functional by design via <code>replaceState</code>). If this regresses, check whether a new navigation call was added using <code>replaceState</code> instead of the established <code>pushState</code> pattern.
A newly-added AWS service type doesn't appear correctly in the builder, or won't deploy	Confirm it's registered in <i>all three</i> places that must stay in sync: <code>src/data/awsServices.ts</code> (catalog entry/fields/defaults), <code>src/utils/resourceRequirements.ts</code> (required-field schema), and <i>both</i> Terraform generators — <code>src/utils/exportTerraform.ts</code> (frontend preview) and the backend's <code>terraformGenerator.js</code> (actual deploy). These two generators are maintained in parallel by hand and have no shared source of truth.
Deployment stuck in "deploying"/"destroying" for a long time	<code>DeploymentsPage</code> 's stuck-deployment detection (Ch. 2.3) and force-destroy dialog; backend <code>terraformDeploymentRunner.js</code> logs for the actual <code>terraform</code> process output.

End of document. This reference reflects a point-in-time investigation of the codebase and should be re-verified against the actual source before being relied on for anything beyond initial orientation — the codebase will continue to change after this was written.